Hitesh Rathore (HR) Notes
https://www.linkedin.com/in/hitesh-rathore-835279305/

JAVASCRIPT STRINGS

DEFINITION

A String is a data type used to represent textual information.

WHAT IS A STRING?

"A String is a sequence of characters."

EXAMPLE

"Hitesh"

H	i	t	e	s	h
↓	↓	↓	↓	↓	↓
Character	Character	Character	Character	Character	Character

THEREFORE

"Hitesh"

=

A sequence of characters

EXAMPLES OF STRINGS

"Hello" "JavaScript" "123" "true" ""

QUESTION	OUTPUT	QUESTION	OUTPUT
`typeof 123` →	"number"	`typeof "123"` →	"string"

IMPORTANT QUESTION

Is "123" a Number?

ANSWER: NO

WHY?

Because quotes exist.

EXAMPLE

123	"123"
↓	↓
Number	String
123	"123"

STRING AS A PRIMITIVE TYPE

Question: Primitive or Non-Primitive?

Answer: Primitive

Why?

Because String directly represents its own value.

NOTE

Some people say that Primitive means Immutable and Non-primitive means mutable =======>
But the meaning of primitive is not immutable, and to think that using two different words means the same is completely wrong.
i.e. the matter is that:
Primitive means Immutable &&
Non-primitive means mutable
i.e. the definition of primitive(P)/non-primitive(NP) is something else, not immutable and mutable.

Primitive: directly represents its own value.

Non-Primitive: directly don't represents its own value.

i.e. Primitive and Non-Primitive are related to memory management.

EXAMPLE

```
let name = "Hitesh";
```

Memory

"Hitesh"

VARIABLE STORES

"Hitesh"

directly.

IMPORTANT PROPERTY

Strings are:

Immutable

MEANING

Once a String is created, its characters cannot be modified.

i.e (actual data me kuch bhi changes nhi kr skte)

NOTE

We will study immutability in detail later.

STRING DECLARATION METHODS

JavaScript provides multiple ways to create Strings.

METHOD 1 ------ SINGLE QUOTES

`'`

```
let name = 'Hitesh';
```

METHOD 2 ------ DOUBLE QUOTES

`"`

```
let name = "Hitesh";
```

QUESTION
Difference?

ANSWER
Practically none.
Both create String primitives.

QUESTION
'Hello'
===
"Hello"
?

ANSWER
true

WHY?
Quotes are not part of the String value.
Both represent the same String.

Q1) I want this output : "Hello World"

ans1) `console.log('"Hello World"');`

Q2) I want this output : 'Hello World'

ans2) `console.log("'Hello World'");`

↳ that's why there are two different ways
SINGLE QUOTES and DOUBLE QUOTES

METHOD 3 ----- TEMPLATE LITERALS

`` ` ``

Uses: Backticks

SYNTAX

```
let name = `Hitesh`;
```

IMPORTANT

Backtick: `` ` `` is different from: `'` and `"`

TEMPLATE LITERALS (BASICS)

QUESTION
Why were Template Literals introduced?

OLD WAY

```
let name = "Hitesh";
let msg = "Hello " + name;
```

OUTPUT

```
Hello Hitesh
```

PROBLEM
As data grows,
String concatenation becomes hard to read.

EXAMPLE

```
let name = "Hitesh";
let age = 19;
let msg =
  "Name: " + name +
  ", Age: " + age;
```

SOLUTION
Template Literals

EXAMPLE

```
let name = "Hitesh";
let age = 19;
let msg =
  `Name:  ${name}, Age:  ${age}`;
```

OUTPUT

```
Name: Hitesh, Age: 19
```

IMPORTANT TERM	CALLED
`${ }`	Interpolation

MEANING
Insert JavaScript expressions inside a String. `${ }`

EXAMPLE

```
let a = 10;
let b = 20;
`${a + b}`
```

OUTPUT

```
30
```

IMPORTANT OBSERVATION
Anything inside: `${ }` is evaluated first.

EXAMPLE	OUTPUT
`` `Result = ${10 + 20}` ``	Result = 30

MULTI-LINE STRINGS

QUESTION: Can Strings span multiple lines?

USING QUOTES

	RESULT
"Hello World"	⊗ Syntax Error

<or> "Hello\nworld"

USING BACKTICKS

	OUTPUT
\`Hello World\`	Hello World

STRING LENGTH

QUESTION:
How does JavaScript know how many characters exist in a String?

ANSWER:
.length

EXAMPLE

```
let name = "Hitesh";
console.log(name.length);
```

OUTPUT

```
6
```

VISUALIZATION

H	i	t	e	s	h
1	2	3	4	5	6

LENGTH 6

EMPTY STRING

""

QUESTION: Length?

EXAMPLE

```
"".length
```

OUTPUT

```
0
```

SPACE CHARACTER

" "

QUESTION: Length?

EXAMPLE

```
" ".length
```

OUTPUT

```
1
```

WHY?
Because space is also a character.

★ OLYMPIAD CHALLENGE ★
DESIGN YOUR OWN SURROGATE SYSTEM

A newly formed computer company claims that UTF-16 is not optimal.
Their engineers decide to create a new system:

UTF-X

You are one of the engineers.
Your task is to design the surrogate system.

INFORMATION-1
Every Code Unit contains:
16 Bits

INFORMATION-2
The company wants to support:
U+0000
to
U+10FFFF

INFORMATION-3
BMP characters
(U+0000 → U+FFFF)
must still use:
1 Code Unit

INFORMATION-4
Supplementary characters
(U+10000 → U+10FFFF)
may use:
2 Code Units

INFORMATION-5
Engineers reserve:
D800 → DFFF
for surrogate purposes.

Q1 Why can't a supplementary character be stored directly in one Code Unit?
Prove mathematically.

Q2 How many bits are required to represent:
U+10FFFF ?

Q3 Supplementary characters start at:
U+10000
Why do we first subtract:
U+10000
before creating surrogate pairs?
Prove mathematically that this removes unnecessary leading values.

Q4 After subtracting:
U+10000
what is the largest possible value?
Show the complete calculation.

Q5 Using your answer from Question-4, determine exactly how many bits are needed to represent every supplementary character.

Q6 Suppose your team decides:

High Surrogate stores:	Low Surrogate stores:
12 Bits	8 Bits

instead of 10 Bits + 10 Bits.
Will every supplementary character still be representable?
Justify your answer.

Q7 Suppose your team decides:
8 Bits + 8 Bits
instead of 10 Bits + 10 Bits.
Determine:
Maximum representable supplementary characters.

Q1 Answer:

- A single Code Unit has 16 bits → max value = $2^{16} - 1 = 65,535$
- But supplementary characters go up to U+10FFFF = 1,114,111.
- Since 1,114,111 > 65,535, they cannot be stored in one Code Unit.
- Therefore, mathematically impossible.

Q2 Answer:

U+10FFFF = 1,114,111 (decimal)
$2^{20} = 1,048,576 < 1,114,111$
$2^{21} = 2,097,152 > 1,114,111$
Hence, 21 bits are required.

Q3 Answer:

- Subtracting U+10000 (65,536) shifts the range so it starts from 0.
- This removes the unnecessary leading "1" in binary.
- It reduces the number of bits needed.
- Hence, it eliminates redundant leading values.

Q4 Answer:

Largest value after subtracting U+10000:
0x10FFFF – 0x10000
= 0x10FFFF – 0x010000
= 0x0FFFFF
= 1,048,575 (decimal)

Q5 Answer:

- largest value = 1,048,575
- $2^{20} = 1,048,576 > 1,048,575$
- $2^{19} = 524,288 < 1,048,575$
- Therefore, 20 bits are needed.

Q6 Answer:

- High Surrogate = 12 bits, Low Surrogate = 8 bits
- Total = 12 + 8 = 20 bits
- Needed bits = 20 (from Q5)
- Therefore, YES, every supplementary character is representable.

Q7 Answer:

- 8 bits + 8 bits = 16 bits total
- Maximum representable values = $2^{16} - 1 = 65,535$
- But required supplementary characters = 1,048,576
- Therefore, not enough.

Maximum representable supplementary characters = 65,535

CHARACTER ACCESS

QUESTION:
How does JavaScript identify individual characters?

ANSWER:
Using indexes.

EXAMPLE

`"Hitesh"`

INDEXES

```
H  i  t  e  s  h
0  1  2  3  4  5
```

IMPORTANT RULE
Indexing starts from:
0
---- NOT ----
1

EXAMPLE

```
let name = "Hitesh";
name[0]
```

OUTPUT

`"H"`

EXAMPLE

```
name[3]
```

OUTPUT

`"e"`

QUESTION

```
name[100]
```

OUTPUT

`undefined`

WHY?

Index does not exist.

ALTERNATIVE METHOD

`charAt()`

EXAMPLE

```
name.charAt(0)
```

OUTPUT

`"H"`

QUESTION

```
name.charAt(100)
```

OUTPUT

`""` (empty String)

IMPORTANT DIFFERENCE

`name[100]`		`name.charAt(100)`
OUTPUT	VS	OUTPUT
`undefined`		`""` (empty String)
WHY?		
Index does not exist. So [] returns undefined.		charAt() returns an empty string when index does not exist.

NEGATIVE INDEX

`name[-1]`		`name.charAt(-1)`
OUTPUT	VS	OUTPUT
`undefined`		`""` (empty String)
WHY?		WHY?
JavaScript Strings do not support negative indexing using [].		charAt() returns an empty string when index does not exist.

STRING IMMUTABILITY

One of the most important String concepts.

① CAN WE MODIFY A CHARACTER INSIDE A STRING?

```
0 1 2 3 4 5
H i t e s h
↑
index 0
```

EXAMPLE

```
let name = "Hitesh";

name[0] = "R";

console.log(name);
```

JS

OUTPUT

```
> Hitesh
```

QUESTION

Why didn't it become:

`Ritesh`

?

ANSWER

Strings are immutable.

IMMUTABLE MEANS

Once a String is created, its contents cannot be changed.

② WHAT HAPPENS HERE?

```
let str = "Hello";

str = "Jello";
```

QUESTION

Did JavaScript modify the old String?

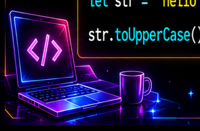
ANSWER

NO

INTERNAL IDEA

Old String `"Hello"` remains unchanged. → JavaScript creates a NEW String `"Jello"` → Variable now points to the new String.

③ INTERVIEW QUESTION



```
let str = "hello";

str.toUpperCase();
```

QUESTION

Did original String change?

ANSWER

NO

WHY?

toUpperCase() returns a NEW String.

EXAMPLE

```
let str = "hello";

let result =
str.toUpperCase();
```

FINAL VALUES

str	result
"hello"	"HELLO"
(unchanged)	(new String)

CHARACTER ENCODING (FIRST PRINCIPLES APPROACH)

Before learning: → UTF-16 → Code Units → Surrogate Pairs → String Internal Storage

We must answer one very important question:

QUESTION

How does a computer store text?

Why Computers Can't Store Characters Directly

Question:
Can RAM store: A B C H i 😀 directly?

ANSWER
✓ NO

WHY?
Because computers understand only: 0 and 1

EXAMPLE

Human understands:	Computer understands:
A	01000001

IMPORTANT OBSERVATION

Humans communicate using:	Computers communicate using:
Characters	Binary Digits

QUESTION
If computers understand only: 0 and 1
Then how can they store: A ?

ANSWER
By assigning a number to the character.

VISUALIZATION
A
↓
Number
↓
Binary

EXAMPLE
A
↓
65
↓
01000001

IMPORTANT IDEA Computers never store characters directly. → They store numbers.

Who decides which character gets which number?

ANSWER
A Character Encoding System

THE BIRTH OF ASCII

QUESTION:
How can all computers agree that: A should be represented by the same number?

PROBLEM
Imagine:

Computer Company A stores:	Computer Company B stores:	Computer Company C stores:
A = 65	A = 120	A = 500

RESULT
Computers would never be able to communicate correctly.

SOLUTION
Create a standard.

THAT STANDARD WAS
ASCII

ASCII
Full Form:
American Standard Code for Information Interchange

OBSERVATION: Ascii characters ko store krne ke liye bnaya gya bcoz agr characters ko hi store krna hai to phir aap apne computer ke liye rules bna do ki A as 1, B = 2, C = 3, i.e computer ka owner khud handle kr skta hai chijo ko to phir global ASCII bnane ki kyu jrunt pdi -> bcoz let computer one ka owner decides that A --> 1, B --> 2 but computer two's owner says: A--> 5, B--> 6 so information exchange nhi kr payenge ab proper way me so
ASCII: American Standard Code for Information Interchange
(look here in full form: for Information Interchange)

FIRST PRINCIPLES DEFINITION
ASCII is a standard mapping table that assigns numbers to characters.

EXAMPLE

A	B	C	a	b	0	1
↓	↓	↓	↓	↓	↓	↓
65	66	67	97	98	48	49
01000001	01000010	01000011	01100001	01100010	00110000	00110001

1 ASCII FUNDAMENTALS

Character → Number Mapping

IMPORTANT OBSERVATION
ASCII does NOT store characters.
ASCII assigns numbers to characters.

ASCII TABLE EXAMPLES

A	Z	a	a	z	0	9
↓	↓	↓	↓	↓	↓	↓
65	90	97	122	43	48	57

HOW MANY CHARCTERS COULD ASCII REPRESENT?
ANSWER
128

WHY?
ASCII used:
7 BITS

7 BITS
Can represent:
0 ···→ 127

TOTAL
128
VALUES

ASCII SUPPORTED

Aa	123	!@#	⚙
English Alphabets	Digits	Basic Symbols	Control Characters

ASCII DID NOT SUPPORT

न	你	あ	ع	😀
Hindi	Chinese	Japanese	Arabic	Emojis

2 THE PROBLEM WITH ASCII

QUESTION	QUESTION	QUESTION
Can ASCII represent: न	Can ASCII represent: 你	Can ASCII represent: 😀
ANSWER NO ✗	ANSWER NO ✗	ANSWER NO ✗

WHY?
ASCII was designed primarily for:
English Text

THE INTERNET ARRIVES
As computers spread worldwide, people wanted to write:

Hindi	Chinese	Japanese	Arabic	Korean	Emojis
न	你	あ	ع	한	😀

QUESTION
Could ASCII handle all these?

ANSWER
NO ✗

3 WHY UNICODE WAS NEEDED

OPTION 1
Each country creates its own system.

India	China	Japan
↓	↓	↓
System A	System B	System C

PROBLEM
All systems become incompatible.

EXAMPLE
A file created in one system may appear as garbage in another.
ÆÐ%$#@!
¤†¤µ≈Ω≈

QUESTION
Can we create one universal system for all languages?

ANSWER
YES ✓

THAT SYSTEM IS
UNICODE

4 UNICODE & CODE POINTS

WHAT IS UNICODE?
Unicode is a universal standard that assigns a unique number to every character in every language.

IMPORTANT IDEA
Every Character
↓
Gets A Unique Number

DOES UNICODE STORE CHARACTERS?
ANSWER
NO ✗

IMPORTANT
Unicode only assigns numbers.

VISUALIZATION
Character
↓
Unicode Number
↓
Code Point
↓
Stored Using
UTF-8 UTF-16 UTF-32
↓
Memory

CODE POINT EXAMPLES

A	😀
↓	↓
Unicode Code Point	Unicode Code Point
↓	↓
U+0041	U+1F600

WHAT DOES U+ MEAN?
ANSWER: Unicode Code Point Notation

UNICODE'S JOB
Character
↓
Unique Number → ITS JOB ENDS HERE

Storage is handled later by:
UTF-8 UTF-16 UTF-32

ASCII vs UNICODE

ASCII
Small Character Set
↓
Mostly English

VS

UNICODE
Universal Character Set
- ✓ Almost Every Language
- ✓ Symbols
- ✓ Mathematical Characters
- ✓ Emojis

Q1	Can computers store characters directly?	Answer: No
Q2	What do computers actually store?	Answer: Numbers
Q3	What is ASCII?	Answer: A standard mapping table that assigns numbers to characters.
Q4	How many characters could ASCII represent?	Answer: 128
Q5	Why was ASCII insufficient?	Answer: It could not represent all languages and symbols.
Q6	Why was Unicode created?	Answer: To create a universal character system for all languages and symbols.
Q7	Does Unicode store characters?	Answer: No
Q8	What is Unicode's primary job?	Answer: Assign a unique number to every character.
Q9	What is a Code Point?	Answer: The unique number assigned to a character by Unicode.
Q10	What comes after Unicode?	Answer: UTF-8 UTF-16 UTF-32 for actual storage.

FROM UNICODE TO UTF-16

INTRODUCTION

Character
↓
Unicode
↓
Code Point
↓
Need For Encoding
↓
UTF-8 UTF-16 UTF-32
↓
</> JavaScript uses UTF-16.

Question:
Unicode gives every character a Code Point.

But how is that Code Point actually stored inside memory?

PART 1 UNDERSTANDING UNICODE PROPERLY

WHAT UNICODE ACTUALLY DOES

UNICODE
GIANT DICTIONARY

Unicode is a giant dictionary.

Its job:
Assign a unique number to every character.

EXAMPLES:

A →	65
ह →	2361
😄 →	128522

IMPORTANT:
- Unicode does NOT store anything in memory.
- Unicode only assigns Code Points.

PART 2 CODE POINTS

Unicode usually writes numbers in hexadecimal.

A	ह	😄
↓	↓	↓
65 (Decimal)	2361	128522
↓	↓	↓
41 (Hex)	0939	1F60A
↓	↓	↓
U+0041	U+0939	U+1F60A
Meaning: Unicode Code Point 0041		

IMPORTANT:

Character ≠ Code Point

PART 3 WHY UNICODE IS NOT ENOUGH

Question:
Unicode assigned:
A → U+0041
Fine.

But how do we store U+0041 in RAM?

✗ Unicode doesn't answer this.

Therefore:
Need For Encoding.

Code Point
↓
Binary
↓
Memory

ENCODING

Encoding converts:

Code Point → Binary → Memory
U+0041 → 01010101 → 0000

Examples:
UTF-8 UTF-16 UTF-32

Our focus:
UTF-16

PART 4 WHAT IS UTF-16

UTF
↓
Unicode Transformation Format

16
↓
16 Bits

Definition:
UTF-16 is an encoding format that stores Unicode characters using 16-bit units.

UTF-16

PART 5 BITS FOUNDATION

Question:
What is a Bit?

Answer:
Smallest unit of information.

Possible values:
0 or 1

1 Bit	4 Bits	8 Bits	16 Bits
↓	↓	↓	↓
0 or 1	0000 → 1111	0000000	0000000000000000
		↓	↓
0 1	0000--1111	11111111	1111111111111111
		00000000	0000000000000000
		11111111	1111111111111111

Question:
How many values can 16 bits represent?

Answer:
2^{16}
↓
65536

Range:
0 → 65535

PART 6 CODE UNITS

Question:
What does UTF-16 actually store? ?

Answer:
Code Units

Definition:
1 Code Unit
=
16 Bits

Visualization:
15 ------------------------- 0
1111111111111111
[16 Bits]

IMPORTANT
⚠ JavaScript's length counts Code Units
NOT Characters.
JS

PART 7 NORMAL CHARACTER STORAGE

Example:
A
↓
Unicode
↓
U+0041
↓
65

Question:
Can 65 fit inside:
0 → 65535 ?

Answer:
YES ✓

Therefore:
Only one Code Unit needed.

Visualization:
A
↓
65
↓
1 UTF-16 Code Unit

Result:
"A". length
1

PART 8 THE BIG PROBLEM

Now consider:
😄
↓
U+1F60A
↓
128522

Question:
Can 128522 fit inside:
0 → 65535 ?

Answer:
NO ✗

Problem:
One Code Unit
is not enough.

PART 9 SURROGATE PAIRS

Solution:
Use two Code Units.

Definition:
A Surrogate Pair is two UTF-16 Code Units that represent one Unicode Code Point.

Visualization:
😄
↓
Code Unit 1
*
Code Unit 2

Important:
One Character
↓
One Code Point
↓
Two Code Units

PART 10 WHY SURROGATE PAIRS EXIST

Question:
Why not simply use 32 bits for everything?

Answer:
Waste of memory.
Most common characters fit inside 16 bits.

RAM

Therefore:
1 Code Unit for most characters.
↓
2 Code Units only when needed.

PART 11 ACTUAL UTF-16 SURROGATE ALGORITHM

Example:
😊
↓
U+1F60A

Step 1
Subtract:
10000 = 2^{16}

Reason:
Surrogate pairs are only used for:
U+10000 →
U+10FFFF

Calculation:
1F60A
− 10000
= F60A

Step 2
Convert to Binary
F60A
↓
1111011000001010

Step 3
Pad to 20 bits
↓
↓
00001111011000001010

Step 4
Split into:
10 bits ↓ 0000111101
10 bits ↓ 1000001010

Step 5
High Surrogate
Prefix: 110110
*
0000111101
↓
1101100000111101
↓
D83D

Step 6
Low Surrogate
Prefix: 110111
*
1000001010
↓
1101111000001010
↓
DE0A

Final Result:
😊
↓
D83D DE0A

Memory:
1101100000111101
1101111000001010

PART 12 JAVASCRIPT LENGTH

Question:
What does .length count?
?

Answer:
UTF-16
Code Units

NOT
Characters.

Examples:
"A".length
↓
1

"ह".length
↓
1

"😄".length
↓
2

Exason:
😄
↓
D83D DE0A
↓
2 Code Units

PART 13 TRICKY EXAMPLE

"A😄B"

Visible Characters:
3

Storage:
A → 😄 → B
→ 1 Unit → 2 Units → 1 Unit

Total:
4 Units

Therefore:
"A😄B".length
↓
4

PART 14 STRING INTERNAL STORAGE

Example:

```
let str = "ABC";
```

Important:
JavaScript Strings are defined as
Sequences of
UTF-16 Code Units
NOT Characters.

Conceptual Storage:
A → B → C
↓ ↓ ↓
1 Unit 1 Unit 1 Unit

Total:
3
UTF-16 Code Units

0041 → 0042 → 0043
(1 Unit) (1 Unit) (1 Unit)

PART 15 ADVANCED NOTE

Visible Character Count 1 ≠ Code Point Count U+ Multiple ≠ UTF-16 Code Unit Count Multiple

Example:	Visible Characters:	Code Points:	Code Units:
👨👩👩 A B C	1	Multiple	Multiple

Therefore: .length can be much larger than 2.

FINAL MENTAL MODEL

A Character → Unicode → U+ Code Point → UTF-16 Encoding → Code Units → 0101 1010 Binary → Memory

UTF-16 STRUCTURE

UTF-16 is a variable-length Unicode encoding that represents characters using:

1 Code Unit or 2 Code Units

IMPORTANT
1 Code Unit
=
16 Bits

UTF-16 CHARACTER TYPES

1) 1-CODE-UNIT CHARACTER
A

2) 2-CODE-UNIT CHARACTER (Surrogate Pair)
😀

1-CODE-UNIT CHARACTER

Used for: BMP Characters (Basic Multilingual Plane)

UNICODE RANGE
U+0000 to U+FFFF

STRUCTURE
xxxxxxxxxxxxxxxxxx
(16 Bits)

EXAMPLES

A	ह	₹
↓	↓	↓
U+0041	U+0939	U+20B9
↓	↓	↓
0000000001000001	0000100100111001	0010000010111001

IMPORTANT OBSERVATION
One Character
↓
One Code Point
↓
One Code Unit
↓
16 Bits ✓

JAVASCRIPT

"A".length ↓	1
"ह".length ↓	1
"₹".length ↓	1

2-CODE-UNIT CHARACTER (SURROGATE PAIR)

Used for:
Emojis Rare Symbols Ancient Scripts Supplementary Characters

UNICODE RANGE
U+10000 to U+10FFFF

STRUCTURE
High Surrogate
110110xxxxxxxxxx
Low Surrogate
110111xxxxxxxxxx

TOTAL
16 Bits + 16 Bits = 32 Bits

EXAMPLE
😄
↓
U+1F60A
↓
D83D DE0A

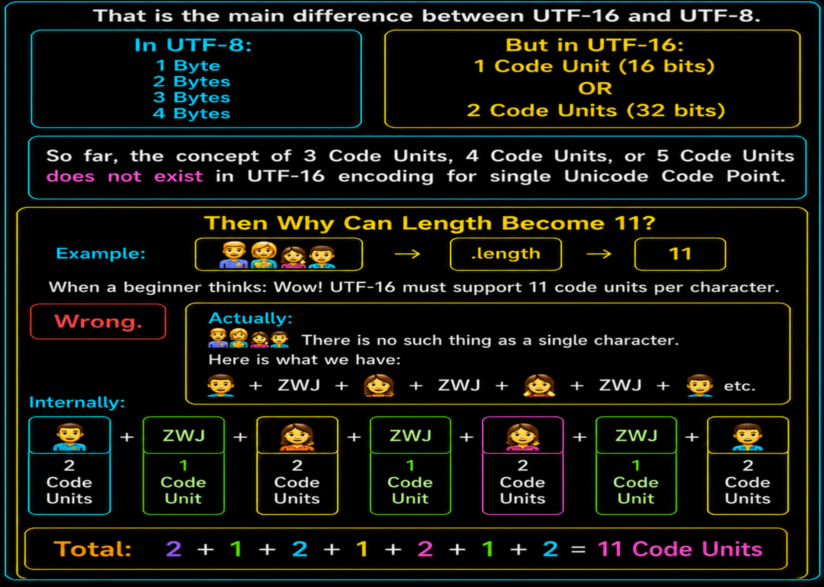
BINARY
D83D
↓
1101100000111101

DE0A
↓
1101111000001010

IMPORTANT OBSERVATION
One Character
↓
One Code Point
↓
Two Code Units
↓
32 Bits ✓

JAVASCRIPT

😄.length ↓ 2	😄.length ↓ 2	😄.length ↓ 2



That is the main difference between UTF-16 and UTF-8.

In UTF-8:
1 Byte
2 Bytes
3 Bytes
4 Bytes

But in UTF-16:
1 Code Unit (16 bits)
OR
2 Code Units (32 bits)

So far, the concept of 3 Code Units, 4 Code Units, or 5 Code Units does not exist in UTF-16 encoding for single Unicode Code Point.

Then Why Can Length Become 11?

Example: 👨‍👩‍👧‍👦 → .length → 11

When a beginner thinks: Wow! UTF-16 must support 11 code units per character.

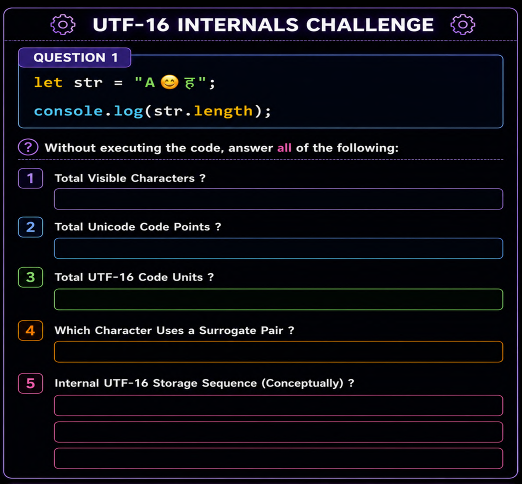
Wrong.

Actually:
👨‍👩‍👧‍👦 There is no such thing as a single character.
Here is what we have:
👨 + ZWJ + 👩 + ZWJ + 👧 + ZWJ + 👦 etc.

Internally:

👨	+	ZWJ	+	👩	+	ZWJ	+	👧	+	ZWJ	+	👦
2 Code Units		1 Code Unit		2 Code Units		1 Code Unit		2 Code Units		1 Code Unit		2 Code Units

Total: 2 + 1 + 2 + 1 + 2 + 1 + 2 = 11 Code Units



UTF-16 INTERNALS CHALLENGE

QUESTION 1

```
let str = "A😊ह";
console.log(str.length);
```

? Without executing the code, answer all of the following:

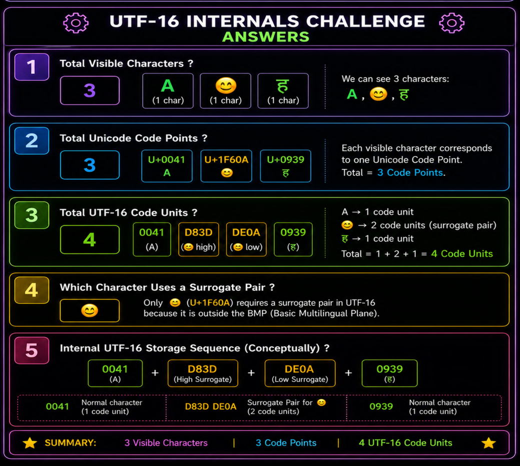
1 Total Visible Characters ?

2 Total Unicode Code Points ?

3 Total UTF-16 Code Units ?

4 Which Character Uses a Surrogate Pair ?

5 Internal UTF-16 Storage Sequence (Conceptually) ?



UTF-16 INTERNALS CHALLENGE
ANSWERS

1 Total Visible Characters ?
3 | A (1 char) | 😊 (1 char) | ह (1 char)
We can see 3 characters:
A , 😊 , ह

2 Total Unicode Code Points ?
3 | U+0041 A | U+1F60A 😊 | U+0939 ह
Each visible character corresponds to one Unicode Code Point.
Total = 3 Code Points.

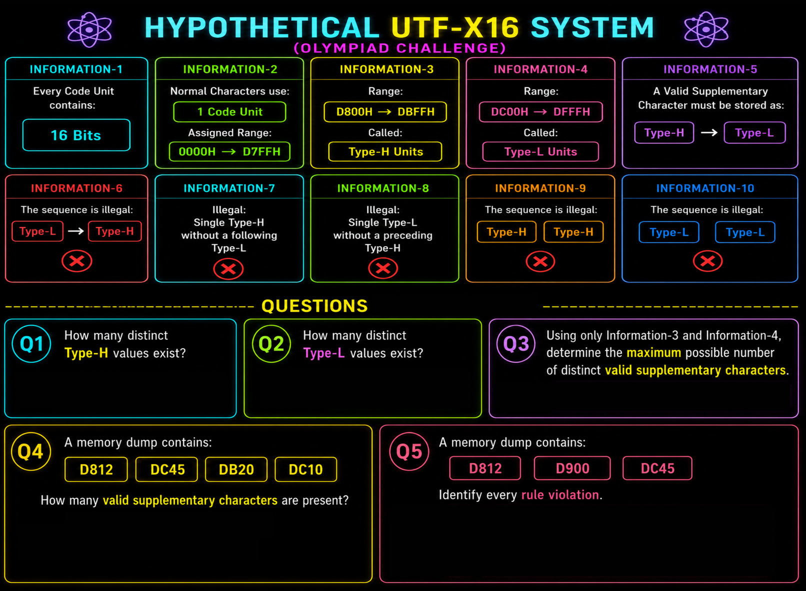
3 Total UTF-16 Code Units ?
4 | 0041 (A) | D83D (😊 high) | DE0A (😊 low) | 0939 (ह)
A → 1 code unit
😊 → 2 code units (surrogate pair)
ह → 1 code unit
Total = 1 + 2 + 1 = 4 Code Units

4 Which Character Uses a Surrogate Pair ?
😊
Only 😊 (U+1F60A) requires a surrogate pair in UTF-16 because it is outside the BMP (Basic Multilingual Plane).

5 Internal UTF-16 Storage Sequence (Conceptually) ?
0041 (A) + D83D (High Surrogate) + DE0A (Low Surrogate) + 0939 (ह)

0041	Normal character (1 code unit)	D83D DE0A	Surrogate Pair for 😊 (2 code units)	0939	Normal character (1 code unit)

★ SUMMARY: 3 Visible Characters | 3 Code Points | 4 UTF-16 Code Units ★



HYPOTHETICAL UTF-X16 SYSTEM
(OLYMPIAD CHALLENGE)

INFORMATION-1
Every Code Unit contains:
16 Bits

INFORMATION-2
Normal Characters use:
1 Code Unit
Assigned Range:
0000H → D7FFH

INFORMATION-3
Range:
D800H → DBFFH
Called:
Type-H Units

INFORMATION-4
Range:
DC00H → DFFFH
Called:
Type-L Units

INFORMATION-5
A Valid Supplementary Character must be stored as:
Type-H → Type-L

INFORMATION-6
The sequence is illegal:
Type-L → Type-H
✗

INFORMATION-7
Illegal:
Single Type-H without a following Type-L
✗

INFORMATION-8
Illegal:
Single Type-L without a preceding Type-H
✗

INFORMATION-9
The sequence is illegal:
Type-H Type-H
✗

INFORMATION-10
The sequence is illegal:
Type-L Type-L
✗

QUESTIONS

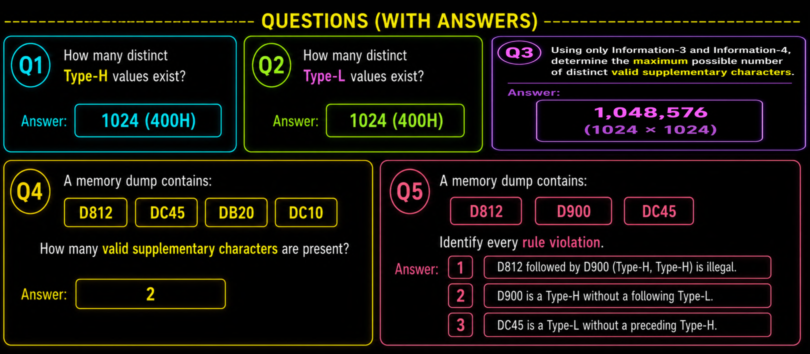
Q1 How many distinct Type-H values exist?

Q2 How many distinct Type-L values exist?

Q3 Using only Information-3 and Information-4, determine the maximum possible number of distinct valid supplementary characters.

Q4 A memory dump contains:
D812 DC45 DB20 DC10
How many valid supplementary characters are present?

Q5 A memory dump contains:
D812 D900 DC45
Identify every rule violation.



QUESTIONS (WITH ANSWERS)

Q1 How many distinct Type-H values exist?
Answer: 1024 (400H)

Q2 How many distinct Type-L values exist?
Answer: 1024 (400H)

Q3 Using only Information-3 and Information-4, determine the maximum possible number of distinct valid supplementary characters.
Answer:
1,048,576
(1024 × 1024)

Q4 A memory dump contains:
D812 DC45 DB20 DC10
How many valid supplementary characters are present?
Answer: 2

Q5 A memory dump contains:
D812 D900 DC45
Identify every rule violation.
Answer:
1 D812 followed by D900 (Type-H, Type-H) is illegal.
2 D900 is a Type-H without a following Type-L.
3 DC45 is a Type-L without a preceding Type-H.